Vanna Solana Copilot / MCP — Answers and Developer Handoff

Prepared: 28 September 2026. Scope: vanna-credit-layer + vanna-credit-backend.

This answers all eight Part A questions and all nine Part B wrapper groups in “Solana Questions MCP.pdf”. The backend is under active development. Existing backend files are implementation references, not a finished API contract or a production-readiness claim. Backend-Solana-v2 is outside this handoff's target scope.

Status labels: Verified means observed in the current local source; Proposed means an implementation recommendation; Pending means a team/deployment decision that cannot be established from source. The PDF is a questionnaire and proposed specification; its assumptions were checked against the code.

Executive answer

Build the Copilot around the current Anchor program and its matching IDL now. Keep chain reads, unsigned transaction construction, oracle preparation and wallet execution behind separate interfaces so the unfinished backend can be connected later. Do not make development depend on a signing endpoint that does not yet exist.

Current source program ID: H37YVBG3eGtz89ofd7iDLc2PJgn12778392XeU2QJcGC. Current development network: a local Surfpool mainnet fork. Current backend asset catalogue: SOL, USDC, USDT, JitoSOL and JupSOL. These are development facts, not confirmation of the production launch network or final asset list.

Important corrections to the supplied PDF:

• Current margin arrays contain 24 entries each, not 8.

• user_deposit_and_borrow is absent from the current program. Compose deposit and borrow instructions instead.

• Reserve interest uses a polynomial RateCurve, not base/slope1/slope2/optimal-utilization fields.

• Kamino and Jupiter integrations exist through margin_execute; they must be considered when valuing a margin account.

• JupSOL's configured feed ID is a synthetic fork sentinel, not a production Hermes feed in this implementation.

• Margin vaults are associated token accounts. They are PDAs of the Associated Token Program, not custom Vanna PDAs; their token-program variant matters.

• The current backend transaction helper signs and broadcasts. It cannot be exposed directly as an unsigned-only MCP builder.

Part A — Answers to the eight build questions

A1. Canonical program ID, IDL and launch cluster

Verified: declare_id!, Anchor.toml, script PDA helpers and the matching backend default all identify H37YVBG3eGtz89ofd7iDLc2PJgn12778392XeU2QJcGC.

Use vanna-credit-layer/target/idl/vanna_lending.json as the generated contract interface. The current copy at vanna-credit-backend/idl/vanna_lending.json is byte-for-byte identical. Both contain 27 instructions. SHA-256 for both files:

b438bfdd66281209440c93cfe0f2f1bbd7bdc65cfa2fccd7b652215851cd32d4

The layer README still mentions BZ812nUv4Qhr2p1JVgmoJGjYTGk1brAXckyhFSCNH3Zg; that statement is stale relative to the working tree. Do not use it to override the current source and IDL.

Verified development target: Anchor provider is localnet; scripts default to http://127.0.0.1:8899; backend WebSocket default is ws://127.0.0.1:8900. The scripts' historical DEVNET_RPC_URL name does not mean public Solana devnet. They operate against a Surfpool mainnet fork with mainnet mint addresses.

Pending: public devnet/testnet/mainnet launch choice, hosted RPC/WS URLs, and confirmation that the deployed binary matches this source/IDL. No live deployment verification was performed for this document. Matching local IDLs does not prove a deployed program match. Before connecting writes, pin the cluster and program ID, verify the executable deployment and its build provenance, and validate account layouts against that deployment.

Sources: layer Anchor.toml, programs/vanna_lending/src/lib.rs, scripts/src/pda.ts, scripts/src/devnet-env.ts; backend lib/solana/config.ts, idl/vanna_lending.json.

A2. Final asset list and discovery

Verified current development baseline: the backend's TOKENS catalogue and the fork bootstrap select SOL/wSOL, USDC, USDT, JitoSOL and JupSOL. All five use classic SPL Token. SOL is displayed as native SOL in the wallet, but the protocol asset is wSOL.

The following are script registration defaults, not a read of current on-chain configuration. Read AssetConfig at runtime for actual risk parameters, flags, decimals, token program and price source. Stored LTV and liquidation-threshold bps are compatibility fields; they do not weight the current health-factor calculation. Liquidation bonus remains active.

SOL / wSOL

Mint: So11111111111111111111111111111111111111112

Decimals: 9. LTV: 7000 bps. Liquidation threshold field: 8000 bps. Liquidation bonus: 500 bps.

Configured feed ID: ef0d8b6fda2ceba41da15d4095d1da392a0d2f8ed0c6c7bc0f4cfac8c280b56d

Oracle status: configured Pyth feed; live availability and subscription entitlement must be checked for the launch environment.

USDC

Mint: EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v

Decimals: 6. LTV: 8000 bps. Liquidation threshold field: 8500 bps. Liquidation bonus: 500 bps.

Configured feed ID: eaa020c61cc479712813461ce153894a96a6c00b21ed0cfc2798d1f9a9e9c94a

Oracle status: configured Pyth feed; live availability and subscription entitlement must be checked for the launch environment.

USDT

Mint: Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB

Decimals: 6. LTV: 8000 bps. Liquidation threshold field: 8500 bps. Liquidation bonus: 500 bps.

Configured feed ID: 2b89b9dc8fdf9f34709a5b106b472f0f39bb6ca9ce04b0fd7f2e971688e2e53b

Oracle status: configured Pyth feed; live availability and subscription entitlement must be checked for the launch environment.

JitoSOL

Mint: J1toso1uCk3RLmjorhTtrVwY9HJ7X8V9yYac6Y7kGCPn

Decimals: 9. LTV: 6500 bps. Liquidation threshold field: 7500 bps. Liquidation bonus: 500 bps.

Configured feed ID: 67be9f519b95cf24338801051f9a808eff0a578ccb388db73b7f6fe1de019ffb

Oracle status: configured Pyth feed; live availability and subscription entitlement must be checked for the launch environment.

JupSOL

Mint: jupSoLaHXQiZZTSfEWMTRRgpnyFm8f6sZdosWBjx93v

Decimals: 9. LTV: 6500 bps. Liquidation threshold field: 7500 bps. Liquidation bonus: 500 bps.

Configured feed ID: 6cf9d0108cc803dded4edde0a18ad3b01cfca1a3f89c1f3a65f9c5a2a6ccc0c1

Oracle status: synthetic sentinel, fork-only in this implementation; no production Hermes mapping established.

Pending final asset decision: no evidence establishes whether more assets will be added before Copilot ships. Additional script-supported assets are listed in Appendix A; they are not automatically part of the current backend launch list.

Proposed discovery design: retain a small static catalogue for symbols/icons, but obtain authoritative assets and parameters from validated on-chain AssetConfig accounts. Build the asset_index → AssetConfig → mint mapping from chain state. Decode with the matching IDL and verify owner, discriminator and PDA. A static catalogue alone must never cause an active collateral or debt position to be omitted from health checks. Unknown positions should stop the action with an explicit unsupported-asset error.

There is no separate address-resolution registry contract. There are asset-config accounts and an Integration whitelist account type. Discovery does not require adding another contract.

The bootstrap also registers five Kamino receipt assets for these underlyings. They are collateral-only and use the underlying feed plus a Kamino reserve exchange rate; they are not independent Pyth-priced tokens. Include them in health/position reads even when the initial Copilot does not expose farm execution. Appendix B gives their addresses.

Sources: backend lib/solana/tokens.ts; layer scripts/bootstrap-fork.sh, scripts/src/devnet-env.ts, scripts/src/devnet.ts, scripts/src/kamino.ts, state/asset_config.rs.

A3. Signing/broadcast service interface

Verified: no dedicated, finalized Copilot signing/broadcast HTTP service was found in the target backend. Its current TxSigner interface provides publicKey, signTransaction and signAllTransactions; sendInstructions compiles a v0 transaction, simulates, requests wallet signatures, broadcasts through RPC and confirms. Oversized transactions may require separate lookup-table preparation transactions. Its return value is a confirmed signature string, not an unsigned transaction response.

Pending: exact service URL, HTTP authentication, request/response schema, error schema, fee sponsorship and execution policy. These are backend work in progress. There is no honest final endpoint or auth credential to supply yet.

Proposed interface for parallel development — not implemented or agreed: let MCP builders return the following envelope to a separate wallet/execution adapter. Decimal strings represent all integer amounts, including u64/u128; never serialize them as floating-point amounts.

{

"schemaVersion": "1",

"cluster": "localnet-fork",

"programId": "H37YVBG3eGtz89ofd7iDLc2PJgn12778392XeU2QJcGC",

"feePayer": "<wallet public key>",

"transactions": [{

"id": "action-1",

"dependsOn": [],

"format": "solana-v0",

"encoding": "base64",

"unsignedTransaction": "<serialized transaction>",

"blockhash": "<recent blockhash>",

"lastValidBlockHeight": "<integer>",

"requiredSigners": ["<public key>"],

"simulation": {

"status": "success",

"err": null,

"logs": [],

"unitsConsumed": 0,

"contextSlot": "<integer>"

}

}],

"summary": {

"action": "borrow",

"mint": "<mint public key>",

"amountRaw": "1000000"

}

}

The example is schematic; placeholder values and zero units are not a real simulation. Allow status values success, failed, and not_run_dependency. Never mark a dependent action successful merely because its oracle/LUT preparation transaction has been simulated separately. Where setup must first be confirmed, return the dependency plan and rebuild/re-simulate the action afterward.

The execution adapter can return a proposed {requestId, status, signatures, error} envelope; status should distinguish awaiting approval, submitted, confirmed and failed. Endpoint path and authentication remain TBD. For a wallet-based flow, the execution adapter can live in the app rather than a remote signing service.

Build unsigned transaction bytes using the intended fee payer and blockhash. Hand off signer public keys, transaction dependencies and action details. Expired blockhashes require rebuilding and simulation again. If Pyth preparation introduces temporary signers, design their custody/signing in the executor explicitly; an unsigned JSON envelope must never contain private keys.

Do not call backend sendInstructions, .rpc(), signTransaction or sendRawTransaction from an unsigned-only MCP builder. Reuse instruction construction after separating it from execution.

Sources: backend lib/solana/wallet-bridge.ts, lib/solana/tx.ts, lib/solana/confirmation.ts; current app/api routes.

A4. Pyth Hermes production access

Verified project state: current development price refresh is fork-oriented and can write synthetic receiver-shaped accounts with Surfpool cheatcodes. That is a test-environment mechanism, not authenticated production oracle publication. The layer's Hermes helper currently creates a client without supplying an API token.

Pending: provisioned production key, provider/subscription, request-rate and monthly quota budget, allowed feeds, and whether an existing subscription can be shared. Source code does not establish these commercial settings. No secret values belong in this document.

Proposed: use a server-side PYTH_API_KEY and configurable PYTH_HERMES_URL; provision a separate MCP/backend credential and quota where possible. This is a recommendation, not confirmation that a separate key exists. Batch feed requests and handle authentication/rate-limit failures explicitly; never substitute

fabricated production prices.

Pyth's current upgrade guide says Hermes now requires an API key and documents Bearer authentication. Verify the chosen endpoint and SDK/receiver compatibility before production integration. Source: Pyth Core upgrade guide (https://docs.pyth.network/price-feeds/core/upgrade/preparing).

Sources: layer scripts/src/devnet-pyth.ts; backend lib/solana/server/fork-prices.ts, lib/solana/price-refresh.ts.

A5. Liquidation scope and caller

Verified contract capability: yes. public_liquidate exists and is permissionless at the program interface: the liquidator signs and provides repayment funds; no keeper identity allowlist is enforced by this instruction.

Pending product answer: whether the Copilot exposes liquidation and whether its executor permits only a keeper or any approved wallet. The contract cannot answer that product decision.

Proposed v1: keep public user-Copilot liquidation execution disabled by default; implement health/eligibility reads, and isolate an optional keeper/advanced builder. This is proposed scope, not an approved requirement.

If enabled, require a complete validated health snapshot with debt > 0 and HF <= 1.10 before building. Deny on missing accounts, unsupported price sources, stale prices or read errors. The on-chain instruction remains authoritative. It caps repayment at 50% of the selected debt position, with a one-raw-unit minimum cap for dust, and requires improved health afterward unless debt is zero. Honor min_collateral_out.

Current limitation: debt and seized collateral must use the same token program because the instruction has one shared token_program account. Do not promise arbitrary classic-SPL/Token-2022 liquidation pairs.

Source: layer instructions/account_manager/liquidate.rs, constants.rs, math/health.rs.

A6. Is health factor 1.1 final or provisional?

Verified for this source version: the threshold is a global constant, not per asset. BALANCE_TO_BORROW_THRESHOLD_WAD = 1100000000000000000, with WAD = 10^18.

HF = sum(collateral USD value) / sum(debt USD value). Borrowed funds left in margin vaults count as collateral. Collateral value rounds down; debt and debt-share conversions round up. Project reserve interest to the chain Clock time before valuing debt. Kamino receipts require conversion to underlying units first.

Healthy: zero debt OR HF > 1.10. Liquidatable: debt > 0 AND HF <= 1.10. Equality is liquidatable. Zero-debt health uses the on-chain u128 maximum sentinel; present this as “no debt” rather than unsafe floating-point infinity in JSON.

ltv_bps and liquidation_threshold_bps remain stored/validated but do not change the current health formula. The backend's 1.15 safety buffer is a UI policy, not the liquidation threshold.

Pending: a promise that the threshold will never change. It is fixed in the current implementation and can change through a future program version. Cite the deployed version's constants.rs, math/health.rs and risk_engine.rs; do not treat a frontend constant as authority.

A7. Admin instructions

Proposed answer: no end-user/agent-facing admin wrappers for v1. Read-only protocol/asset/reserve configuration is needed. Operational administration should remain separate. Final scope is a team decision, not derivable from backend completion status.

If a separate admin tool is later required, use the IDL's exact accounts and signer checks. Include the current additional instructions admin_set_asset_price_source, admin_register_integration and admin_set_integration_enabled, which the PDF omits. authority_accept_admin requires the pending admin's signature; it is not an ordinary user action.

Source: layer lib.rs, instructions/admin/.

A8. Faucet support

Verified current backend implementation: a development POST /api/faucet route accepts { "address": "<public key>", "symbol": "USDC" }; omitting symbol funds the current registry tokens. It returns {ok: true, results, note} on success or {error} with an error status. It uses Surfpool cheatcodes and does not produce a normal funding transaction. This route is still part of the in-progress backend.

Proposed scope: optional fork-only development faucet support, disabled for production. Public devnet SOL airdrop and a devnet USDC faucet link are separate features and are not confirmed requirements or implemented equivalents of this fork faucet. Final yes/no remains pending.

Sources: backend app/api/faucet/route.ts, lib/solana/server/fork-faucet.ts.

Part B — Corrected wrapper requirements

The nine names in the PDF are client module boundaries. They are not nine deployed contracts. Use the one vanna_lending program and its matching IDL for instruction data, account ordering, signer and writable flags. Source paths below are relative to vanna-credit-layer/programs/vanna_lending/src/ unless stated otherwise.

B1. ProtocolConfigContract

PDA seeds: ["protocol"]. Read admin, pending_admin, treasury, operating_mode, max_assets_per_margin and next_asset_index, plus layout fields from the IDL. Resolve mode values as Normal=0, BorrowPaused=1, WithdrawOnly=2, Halted=3.

Read support belongs in the initial Copilot. Admin builders are conditional on A7. The PDF's listed protocol/admin instructions exist. Action permissions must follow validation/accounts.rs and each instruction's actual checks; do not infer that a mode name blocks every action. In particular, public_liquidate currently has no ProtocolConfig account, so do not claim it observes a global protocol-mode gate.

Sources: state/protocol_config.rs, instructions/admin/protocol.rs, validation/accounts.rs.

B2. AssetConfigContract

PDA seeds: ["asset", mint]. In addition to the PDF's fields, decode token_program, price_source, price_source_account and price_source_program. PriceSource supports Pyth and KaminoReceipt. Use the actual asset index, not catalogue position or registration assumptions.

Read flags and caps from chain state. Admin registration/update/price-source builders remain separate. Kamino receipts are collateral-only and need a valid source account; do not build a Vanna borrowing pool for every receipt mint.

Sources: state/asset_config.rs, instructions/admin/assets.rs, oracle/valuation.rs.

B3. ReserveContract — Earn

Reserve seeds: ["reserve", mint]; share mint seeds: ["share_mint", mint]. Read the PDF's balance/share fields plus asset_config, last_update_timestamp, rate_curve, reserve factor and status. Replace the obsolete base/slope/optimal-utilization fields with linear_coeff_wad, jump_coeff_wad and rate_multiplier_wad.

Builders: lender_supply(assets: u64, min_shares_out: u64), lender_redeem(shares: u64, min_assets_out: u64), public_refresh_reserve(). Admin reserve builders are conditional on A7. Read share-mint supply for exchange-rate calculations.

Current math, using integer fixed-point arithmetic:

• Utilization u = total_borrow_assets / (accounted_liquidity_assets + total_borrow_assets); zero for an empty pool.

• Annual borrow-rate polynomial = multiplier × [linear × (u + u^32) + jump × u^64]. Coefficients are stored at WAD scale. Mirror borrow_rate_per_second_wad and its rounding for exact results.

• Year length = 31,556,952 seconds. Interest accrues simply within one time interval, using ceil(debt × per-second-rate × elapsed / WAD); repeated accrual changes the base.

• Protocol fee = floor(interest × reserve_factor_bps / 10,000).

• Lender assets = accounted liquidity + accrued borrow assets − accrued protocol fees. Unsolicited vault donations are excluded.

• Redeem output = floor(shares × lender assets / total share supply). Initial supply conversion is 1:1; the initial minimum-share check is 1,000 raw shares.

• Available lender cash = max(accounted liquidity − accrued protocol fees, 0).

• Current backend displayed supply APR = borrow APR × utilization × (1 − reserve factor). Treat this as the existing display convention. Do not relabel APR as APY; a compounded APY needs an explicitly stated compounding assumption.

Use contract math/interest.rs and math/shares.rs as authority, with backend lib/solana/math.ts and protocol-state.ts as implementation references. The PDF's unspecified lib/pool-stats.ts is not the canonical helper for this target backend. Reserve statuses: Active=0, SupplyOnly=1, RepayOnly=2, Frozen=3; evaluate the actual per-action gates.

B4. MarginAccountContract

Seeds: ["margin", authority]; one account per wallet. Read authority, status, collateral_count, debt_count, event_sequence and both [u16; 24] index arrays. Empty slots contain 65535. Scan the arrays and filter the sentinel; do not assume the first count entries are densely packed. The runtime protocol limit is also relevant.

Builders: user_create_margin() and user_close_margin(). Closing requires an empty active-position registry. A zero-debt position can still require an explicit close instruction. Resolve every active index through AssetConfig and include receipts in health reads.

Sources: state/margin_account.rs, instructions/account_manager/account.rs, risk_engine.rs.

B5. CollateralContract

Derive the margin vault as getAssociatedTokenAddressSync(mint, marginPda, true, tokenProgram). Passing the token program explicitly is essential for Token-2022 support. SOL wallet flows need wrap/unwrap handling around wSOL.

Builders: user_deposit_collateral(amount: u64), user_withdraw_collateral(amount: u64, min_health_factor_wad: u128), user_close_collateral_position(). Deposit does not require a price-health check. Withdrawal requires fresh prices and complete remaining accounts, then verifies projected health; the optional caller minimum cannot relax the protocol threshold. Receipt withdrawals need price_source_account.

Transfers may incur Token-2022 fees; derive outcomes from measured received amounts and simulate. Closing collateral requires a zero vault balance. There is no separate CollateralPosition account type in this design.

Source: instructions/account_manager/collateral.rs, validation/token.rs.

B6. DebtPositionContract — Borrow

Seeds: ["debt", margin, reserve]. Read margin_account, reserve and borrow_shares. Current debt = ceil(position borrow shares × accrued reserve borrow assets / reserve borrow shares). Handle the zero-share invariant exactly as the contract does.

Builders: user_open_debt_position(), user_borrow(assets: u64, max_debt_shares: u128), user_close_debt_position(). Open the debt account before the first borrow; close only at zero borrow shares. Borrowed tokens go to the margin vault, not directly to the user's wallet, and count in post-borrow collateral.

PDF correction: user_deposit_and_borrow does not exist in this source or matching IDL. A Copilot “deposit and borrow” action should compose margin/debt setup when required, deposit collateral and borrow into one transaction where size/compute allow. Build the borrow's remaining accounts for the state after earlier

instructions, including newly registered collateral. If setup requires multiple transactions, report their dependencies and do not advertise the full flow as atomic.

Sources: instructions/account_manager/borrow.rs, math/shares.rs, lib.rs.

B7. RepayContract

Builders: user_repay_from_margin(max_assets: u64, repay_all: bool) and public_repay_from_wallet(max_assets: u64, repay_all: bool).

The first consumes tokens held by the margin account and requires its authority; the second allows a payer to repay another margin account from the payer's wallet. Both accrue interest; repayment/share burning must match actual received tokens and rounding. These instructions do not need the borrow/withdraw price scan.

A repay-all flow may gross up Token-2022 transfer fees. The margin repayment path can be balance-limited; re-read remaining debt instead of promising that setting repay_all always closes it. Debt-position closure is a separate instruction.

Source: instructions/account_manager/borrow.rs.

B8. LiquidationContract

Conditional on A5. Builder: public_liquidate(max_repay_assets: u64, min_collateral_out: u64). Supply liquidator signer/source/destination accounts, margin account, debt reserve and position, both asset configurations/mints/prices, optional receipt source and all other open positions.

There is no dedicated on-chain health view instruction. Compute a conservative pre-check from validated complete state and accrued debt, then simulate. Enforce A5's close-factor and same-token-program limitations; do not imply an off-chain pre-check guarantees transaction success after prices/state change.

Source: instructions/account_manager/liquidate.rs.

B9. OracleContract — Pyth

Configured receiver owner: rec5EKMGg6MxZYaMdyBfgwp4d5rB9T1VQH5pJv5LtFJ.

Configured push-oracle derivation program: pythWSnswVUd12oZpeFP8e9CVaEqJg25g1Vtc2biRsT. The script uses shard 0 (two zero bytes) plus the 32-byte feed ID for the feed-account PDA. Do not confuse a hex feed ID, an oracle account address and the oracle owner's program ID.

Use the compatible receiver SDK to decode/validate owner, discriminator/layout, verification level, feed ID, positive price, publish time/staleness and confidence against AssetConfig. Registration defaults are max_price_age_secs=3600 and max_confidence_bps=1000, but live asset settings are authoritative. Fixed offsets copied from the fork helper are not a general decoder for every verification variant.

Automatically prepare fresh oracle inputs inside borrow, withdraw, composed deposit/borrow, liquidation and margin_execute builders. Cover every relevant position, not just the requested asset. Fresh already-posted push accounts can be reused when validated; a new post is not inherently necessary on every call.

For production pull updates, obtain authenticated Hermes data and build the receiver update instructions followed by the lending action. The unsigned MCP returns the transaction or dependency plan; the separate approved execution layer signs/posts it. Do not silently broadcast oracle updates from an unsigned-only wrapper. If an action depends on a separately confirmed update, simulate the dependent action after that update exists.

For the current fork, expose an explicit development oracle adapter. JupSOL and optional PreStocks use synthetic feeds here; that mechanism must not be presented as a ready production oracle integration. Receipt valuation also needs its validated Kamino source reserve.

Sources: oracle/pyth.rs, oracle/valuation.rs, layer scripts/src/devnet-pyth.ts, backend lib/solana/pyth.ts. External reference: Pyth Solana integration (https://docs.pyth.network/price-feeds/core/use-real-time-data/pull-integration/solana).

Additional module required by the current contract

ExternalIntegration / MarginExecution

The PDF's “no swap/farm/external lending integration” note is obsolete. margin_execute(data: Vec<u8>, cpi_account_count: u16, min_received: u64) exists, with compiled Kamino and Jupiter adapters and an on-chain whitelist PDA ["integration", external_program_id].

Decode Integration.program_id, adapter and enabled. The first cpi_account_count remaining accounts describe the external call; the remaining accounts cover Vanna health validation. Follow the adapter's exact allowed calls and role/account checks. Post-call margin health must pass. Kamino receipt collateral must be valued using reserve exchange rates; it cannot be treated as an ordinary 1:1 token.

Proposed build sequencing: ship core lending/margin reads and builders first, then expose controlled swap/farm actions as backend workflows stabilize. Receipt-aware health reads are necessary from day one if existing accounts can contain receipts. Full external-call execution scope remains a product decision; do not offer arbitrary program invocation through Copilot.

Sources: instructions/account_manager/exec.rs, adapters/kamino.rs, adapters/jupiter.rs, state/integration.rs; layer scripts/src/integrations-fork.ts; backend lib/solana/farm.ts, swap.ts.

Remaining-account construction — shared implementation rule

Use the margin arrays' stored order. First process active collateral positions, excluding only the collateral positions already valued through named instruction accounts. Each group is [asset_config, margin_vault, price_update], with a fourth price_source_account for non-Pyth collateral. Then process active debt positions, excluding the named debt position if the instruction does so. Each group is [asset_config, reserve, debt_position, price_update].

Exactly mirror each instruction's exclusions. A mint that is both debt and collateral can appear in both logical groups. Never deduplicate away required groups, sort by symbol, silently skip unknown indices or append arbitrary extra accounts. The risk engine rejects missing, substituted, reordered or extra accounts. Other reserves are accrued in memory for health checks.

Source: risk_engine.rs::scan_and_validate_positions. The script helper scripts/src/devnet-positions.ts is useful for ordering, but its skip-on-unknown behavior should be hardened to an explicit builder error for the Copilot.

Working with the unfinished backend

Can be built now: IDL decoding, PDA derivation, chain-backed discovery, portfolio reads, exact health math, accrual/share calculations, unsigned builders, remaining-account validation and simulation result formatting. Use an RPC adapter directly; do not require future HTTP routes for basic program access.

Reuse only as evolving references: backend lib/solana/program.ts, pdas.ts, protocol-state.ts, math.ts, lend.ts, margin.ts, wallet-bridge.ts and oracle helpers. Separate transaction construction from their signing/sending behavior. Approximate UI health summaries are not keeper or borrow-authorization engines.

Backend work still to finalize: stable execution adapter/API, authentication, transaction approval/status/error contract, production oracle handling, hosted network configuration, launch asset manifest and optional keeper/faucet/admin exposure.

Suggested implementation checks: snapshot the IDL hash; test empty/partial margin arrays and unknown indexes; test debt rounding and exactly HF=1.10; include Kamino receipts and stale prices; simulate composite deposit/borrow with newly active collateral; exercise expired blockhashes and multi-transaction dependencies. Existing layer tests include test_math, test_full_flow, test_liquidation, test_security and external Kamino/Jupiter suites. No program tests or live transactions were run to produce this documentation.

Appendix A — Additional script-supported assets, not confirmed launch assets

These assets exist in the layer scripts but are outside the five-token backend catalogue reviewed here. They use Token-2022 in the script mapping. Parameters below are registration defaults only.

TSLAx

Mint: XsDoVfqeBukxuZHWhdvWHBhgEHjGNst4MLodqsJHzoB

Decimals: 8. LTV: 5500 bps. Liquidation threshold field: 6500 bps. Liquidation bonus: 700 bps.

Configured feed ID: e6da44bff5b8b06897a3739dd331b440d6662595bb862e37046892c568ae3fc0

Oracle status: configured Pyth feed; live availability and subscription entitlement must be checked for the launch environment.

GOOGLx

Mint: XsCPL9dNWBMvFtTmwcCA5v3xWPSMEBCszbQdiLLq6aN

Decimals: 8. LTV: 6000 bps. Liquidation threshold field: 7000 bps. Liquidation bonus: 700 bps.

Configured feed ID: ad519718d387de4f0d7d29ea16a3730ce42e49c59fef6fba6fc9bac477645f6f

Oracle status: configured Pyth feed; live availability and subscription entitlement must be checked for the launch environment.

AAPLx

Mint: XsbEhLAtcf6HdfpFZ5xEMdqW8nfAvcsP5bdudRLJzJp

Decimals: 8. LTV: 5500 bps. Liquidation threshold field: 6500 bps. Liquidation bonus: 700 bps.

Configured feed ID: aaba35e6f33fb973bb2201d48a79ae24795affa6ba8bd50a93dcaf7da0030f36

Oracle status: configured Pyth feed; live availability and subscription entitlement must be checked for the launch environment.

ANTHROPIC

Mint: Pren1FvFX6J3E4kXhJuCiAD5aDmGEb7qJRncwA8Lkhw

Decimals: 9. LTV: 5500 bps. Liquidation threshold field: 6500 bps. Liquidation bonus: 700 bps.

Configured feed ID: e2b7d8199aa16dfcefcb89b6614e0a95c08bc283745412d209d0b3ca099b0721

Oracle status: synthetic sentinel, fork-only in this implementation; no production Hermes mapping established.

OPENAI

Mint: PreweJYECqtQwBtpxHL171nL2K6umo692gTm7Q3rpgF

Decimals: 9. LTV: 5500 bps. Liquidation threshold field: 6500 bps. Liquidation bonus: 700 bps.

Configured feed ID: 1847efd0848d500cd781f05957ca787d64e80cfc7cefd734337fc9858626e634

Oracle status: synthetic sentinel, fork-only in this implementation; no production Hermes mapping established.

Appendix B — Kamino receipt assets

All listed receipt mints use classic SPL Token and 6 decimals in the scripts. Default registration parameters: LTV 5000 bps, liquidation-threshold field 6000 bps, liquidation bonus 500 bps, max confidence 1000 bps and max price age 3600 seconds. Final registration enables collateral and disables borrowing. The configured feed is the underlying asset's feed from A2 or Appendix A; multiply by the validated receipt exchange rate. Symbols below are script keys, not guaranteed on-chain metadata.

ktslax — TSLAx receipt

Receipt mint: 6bZpUNY1qmbvQBgCmfQJUA377X63ATvnpCHYh8hQnfjC

Price-source Kamino reserve: 5iTiczqgUegqA3PpoNpotizMbY9n1sRWr3oL6igKvWuf

Underlying feed ID: e6da44bff5b8b06897a3739dd331b440d6662595bb862e37046892c568ae3fc0

Bootstrap status: supported by scripts; not in current five-asset bootstrap.

kgooglx — GOOGLx receipt

Receipt mint: FL41HF8KMuMmYHxGgHezsa5MLUKmNSsu32cC8Qru7TnB

Price-source Kamino reserve: 4wg6rEkGgHaEuxMduP46C1xFZ24Lnp5YgdNkZAHxFzsN

Underlying feed ID: ad519718d387de4f0d7d29ea16a3730ce42e49c59fef6fba6fc9bac477645f6f

Bootstrap status: supported by scripts; not in current five-asset bootstrap.

kusdc — USDC receipt

Receipt mint: B8V6WVjPxW1UGwVDfxH2d2r8SyT4cqn7dQRK6XneVa7D

Price-source Kamino reserve: D6q6wuQSrifJKZYpR1M8R4YawnLDtDsMmWM1NbBmgJ59

Underlying feed ID: eaa020c61cc479712813461ce153894a96a6c00b21ed0cfc2798d1f9a9e9c94a

Bootstrap status: included in current five-asset bootstrap.

kwsol — SOL / wSOL receipt

Receipt mint: 2UywZrUdyqs5vDchy7fKQJKau2RVyuzBev2XKGPDSiX1

Price-source Kamino reserve: d4A2prbA2whesmvHaL88BH6Ewn5N4bTSU2Ze8P6Bc4Q

Underlying feed ID: ef0d8b6fda2ceba41da15d4095d1da392a0d2f8ed0c6c7bc0f4cfac8c280b56d

Bootstrap status: included in current five-asset bootstrap.

kusdt — USDT receipt

Receipt mint: B8zf4kojJbwgCRKA7rLaLhRCZBGhgAJp8wPBVZZHMhSv

Price-source Kamino reserve: H3t6qZ1JkguCNTi9uzVKqQ7dvt2cum4XiXWom6Gn5e5S

Underlying feed ID: 2b89b9dc8fdf9f34709a5b106b472f0f39bb6ca9ce04b0fd7f2e971688e2e53b

Bootstrap status: included in current five-asset bootstrap.

kjitosol — JitoSOL receipt

Receipt mint: 9ucQp7thL38MDDTSER5ou24QnVSTZFLevDsZC1cAFkKy

Price-source Kamino reserve: EVbyPKrHG6WBfm4dLxLMJpUDY43cCAcHSpV3KYjKsktW

Underlying feed ID: 67be9f519b95cf24338801051f9a808eff0a578ccb388db73b7f6fe1de019ffb

Bootstrap status: included in current five-asset bootstrap.

kjupsol — JupSOL receipt

Receipt mint: 8euYpQt56z9hA2ZcFCQuWK6gAptrLcJuMZ5PtMpUwbuY

Price-source Kamino reserve: DGQZWCY17gGtBUgdaFs1VreJWsodkjFxndPsskwFKGpp

Underlying feed ID: 6cf9d0108cc803dded4edde0a18ad3b01cfca1a3f89c1f3a65f9c5a2a6ccc0c1

Bootstrap status: included in current five-asset bootstrap.

Appendix C — Current instruction inventory

Generated from the matching local IDL. Account metas, discriminators, custom types and optional accounts must come from that same IDL. The inventory is not proof of live deployment.

• admin_collect_protocol_fees(amount: u64)

• admin_initialize_reserve(rate_curve: RateCurve, reserve_factor_bps: u16, supply_cap: u64, borrow_cap: u64, status: u8)

• admin_propose_authority(new_admin: pubkey)

• admin_register_asset(price_feed_id: [u8; 32], max_collateral_per_margin: u64, ltv_bps: u16, liquidation_threshold_bps: u16, liquidation_bonus_bps: u16, max_confidence_bps: u16, max_price_age_secs: u32, collateral_enabled: bool, borrow_enabled: bool)

• admin_register_integration(adapter: AdapterKind)

• admin_set_asset_price_source(price_source: PriceSource, source_program: pubkey)

• admin_set_integration_enabled(enabled: bool)

• admin_set_operating_mode(new_mode: u8)

• admin_update_asset_config(max_collateral_per_margin: u64, ltv_bps: u16, liquidation_threshold_bps: u16, liquidation_bonus_bps: u16, max_confidence_bps: u16, max_price_age_secs: u32, collateral_enabled: bool, borrow_enabled: bool)

• admin_update_reserve_config(rate_curve: RateCurve, reserve_factor_bps: u16, supply_cap: u64, borrow_cap: u64, status: u8)

• authority_accept_admin()

• initialize_protocol(treasury: pubkey, max_assets_per_margin: u8)

• lender_redeem(shares: u64, min_assets_out: u64)

• lender_supply(assets: u64, min_shares_out: u64)

• margin_execute(data: bytes, cpi_account_count: u16, min_received: u64)

• public_liquidate(max_repay_assets: u64, min_collateral_out: u64)

• public_refresh_reserve()

• public_repay_from_wallet(max_assets: u64, repay_all: bool)

• user_borrow(assets: u64, max_debt_shares: u128)

• user_close_collateral_position()

• user_close_debt_position()

• user_close_margin()

• user_create_margin()

• user_deposit_collateral(amount: u64)

• user_open_debt_position()

- user_repay_from_margin(max_assets: u64, repay_all: bool)
- user_withdraw_collateral(amount: u64, min_health_factor_wad: u128)

Pending decisions for the team

1. Confirm launch cluster/RPC and publish the deployment-matched source and IDL version.

2. Freeze the launch asset list; decide whether JupSOL needs a real supported production oracle path or stays fork-only.

3. Finalize execution request/response schemas, authentication and wallet/fee-payer responsibilities.

4. Provision Hermes credentials, feed entitlements and a quota budget.

5. Decide whether liquidation is exposed, and to which executor identities.

6. Confirm admin and faucet product scope, plus swap/farm launch timing.

These decisions do not prevent building the verified contract readers and unsigned transaction builders now. The backend's ongoing work should be integrated through versioned interfaces when those decisions are settled.

Evidence and version notes

The layer repository HEAD was aae7659877d6e1c4ee416fcd815e0562eae8aac0, with existing local modifications including the program ID, bootstrap and script configuration. This document describes the working tree, not that commit alone. The backend also contains ongoing local development files. Program/backend code was not changed for this handoff. No signing service, production credential, deployed-state match or final product decision has been invented.

Primary references are the source paths cited in each section, relative to the two project directories. For state and instruction paths in A2–A8, use vanna-credit-layer/programs/vanna_lending/src/ as the prefix. Exact mint/feed values were extracted from the current scripts. External Pyth documentation was checked on the preparation date; verify again when production integration begins.